

=====

CAPSTONE PROJECT

NOTEBOOK 16: BUILD 10-GENRE PILOT

=====

Purpose:

This notebook builds a broader 10-genre pilot subset from

FMA-Large by extending the validated 8-genre setup with

International and Spoken.

The goal is to:

1. Load labelled FMA-Large tracks with audio present
2. Select 10 top-level genres
3. Inspect available counts by split and genre

4. Build a balanced 10-genre pilot using safe caps

5. Save clean pilot metadata for downstream modelling

=====

In [1]:

```
# =====
# 1. IMPORT LIBRARIES
# =====

import os
import random
import numpy as np
import pandas as pd

SEED = 42
random.seed(SEED)
np.random.seed(SEED)

print("Seeds set to:", SEED)
```

Seeds set to: 42

In [2]:

```
# =====
# 2. LOAD TRACK METADATA
# =====

tracks = pd.read_csv(
    "../data/raw/metadata/tracks.csv",
    header=[0, 1],
    index_col=0
)

tracks.index = tracks.index.astype(int)

print("Tracks shape:", tracks.shape)
display(tracks.head())
```

Tracks shape: (106574, 52)

	comments	date_created	date_released	engineer	favorites	id	information	listens
track_id								
2	0	2008-11-26 01:44:45	2009-01-05 00:00:00	NaN	4	1		6073
3	0	2008-11-26 01:44:45	2009-01-05 00:00:00	NaN	4	1		6073
5	0	2008-11-26 01:44:45	2009-01-05 00:00:00	NaN	4	1		6073
10	0	2008-11-26 01:45:08	2008-02-06 00:00:00	NaN	4	6	NaN	47632
20	0	2008-11-26 01:45:05	2009-01-06 00:00:00	NaN	2	4	"spiritual songs" from Nicky Cook	2710

5 rows × 52 columns

In [3]:

```
# =====
# 3. DEFINE AUDIO PATH FUNCTION
# =====

def get_audio_path(track_id, base_dir="../../data/raw/audio/fma_large"):
    track_id_str = f"{int(track_id):06d}"
    folder = track_id_str[:3]
    return os.path.join(base_dir, folder, f"{track_id_str}.mp3")
```

In [4]:

```
# =====
# 4. FILTER TO LABELLED LARGE TRACKS WITH AUDIO PRESENT
# =====

large_tracks = tracks[tracks[("set", "subset")] == "large"].copy()
large_tracks = large_tracks[large_tracks[("track", "genre_top")].notna()].copy()

large_tracks["audio_path"] = [get_audio_path(idx) for idx in large_tracks.index]
```

```

large_tracks["audio_exists"] = large_tracks["audio_path"].apply(os.path.exists)

large_tracks = large_tracks[large_tracks["audio_exists"] == True].copy()

print("Large labelled tracks with audio:", large_tracks.shape)

print("\nTop available genres:")
print(large_tracks[["track", "genre_top"]].value_counts().head(20))

```

Large labelled tracks with audio: (24598, 54)

Top available genres:

(track, genre_top)

Experimental	8357
Rock	7079
Electronic	3058
Hip-Hop	1351
Folk	1284
Pop	1146
Instrumental	729
Classical	611
International	371
Spoken	305
Jazz	187
Old-Time / Historic	44
Blues	36
Soul-RnB	21
Country	16
Easy Listening	3

Name: count, dtype: int64

In [5]:

```

# =====
# 5. DEFINE THE 10-GENRE PILOT LABEL SPACE
# =====

pilot_genres_10 = [
    "Classical",
    "Electronic",
    "Experimental",
    "Folk",
    "Hip-Hop",
    "Instrumental",
    "Pop",
    "Rock",
    "International",
    "Spoken"
]

pilot_tracks_10 = large_tracks[
    large_tracks[["track", "genre_top"]].isin(pilot_genres_10)
].copy()

print("10-genre pilot candidate shape:", pilot_tracks_10.shape)

```

```
print("\n10-genre counts:")
print(pilot_tracks_10[("track", "genre_top")].value_counts())
```

10-genre pilot candidate shape: (24291, 54)

10-genre counts:
(track, genre_top)
Experimental 8357
Rock 7079
Electronic 3058
Hip-Hop 1351
Folk 1284
Pop 1146
Instrumental 729
Classical 611
International 371
Spoken 305
Name: count, dtype: int64

In [6]:

```
# =====
# 6. CHECK AVAILABLE COUNTS BY SPLIT AND GENRE
# =====

split_genre_counts_10 = (
    pilot_tracks_10
    .groupby([("set", "split"), ("track", "genre_top")])
    .size()
    .unstack(fill_value=0)
)

print("Available counts by split and genre:")
display(split_genre_counts_10)
```

Available counts by split and genre:

(track, genre_top)	Classical	Electronic	Experimental	Folk	Hip- Hop	Instrumental	International	Pop
(set, split)								
test	25	207	860	147	103	135	26	85
training	574	2612	6756	1060	1149	534	310	870
validation	12	239	741	77	99	60	35	191

In [7]:

```
# =====
# 7. CHOOSE TRAINING CAP ONLY
# =====
# We balance ONLY the training split.
```

```
# Validation and test will use all available official examples.

USER_MAX_TRAIN_CAP = 180

safe_train_cap = split_genre_counts_10.loc["training", pilot_genres_10].min()
TRAIN_CAP = min(USER_MAX_TRAIN_CAP, safe_train_cap)

print("Minimum available training count across selected 10 genres:")
print("safe_train_cap =", safe_train_cap)

print("\nUser requested max training cap:")
print("USER_MAX_TRAIN_CAP =", USER_MAX_TRAIN_CAP)

print("\nFinal training cap to be used:")
print("TRAIN_CAP =", TRAIN_CAP)

print("\nValidation and test will use ALL official available examples.")
```

Minimum available training count across selected 10 genres:
safe_train_cap = 184

User requested max training cap:
USER_MAX_TRAIN_CAP = 180

Final training cap to be used:
TRAIN_CAP = 180

Validation and test will use ALL official available examples.

In [8]:

```
# =====
# 8. BUILD 10-GENRE PILOT
# =====
# Training is balanced and capped.
# Validation and test use the full official split.

pilot_parts_10 = []

for genre in pilot_genres_10:
    genre_df = pilot_tracks_10[pilot_tracks_10[("track", "genre_top")] ==
    genre].copy()

    train_df = genre_df[genre_df[("set", "split")] == "training"].sample(
        n=TRAIN_CAP,
        random_state=SEED
    )

    val_df = genre_df[genre_df[("set", "split")] == "validation"].copy()
    test_df = genre_df[genre_df[("set", "split")] == "test"].copy()

    pilot_parts_10.append(train_df)
    pilot_parts_10.append(val_df)
    pilot_parts_10.append(test_df)

pilot_10 = pd.concat(pilot_parts_10).sort_index().copy()
```

```
print("10-genre pilot shape:", pilot_10.shape)

print("\n10-genre pilot counts:")
print(pilot_10[("track", "genre_top")].value_counts())

print("\n10-genre split counts:")
print(pilot_10[("set", "split")].value_counts())
```

10-genre pilot shape: (6329, 54)

10-genre pilot counts:
(track, genre_top)

Experimental	1781
Rock	1546
Electronic	626
Pop	456
Folk	404
Hip-Hop	382
Instrumental	375
Spoken	301
International	241
Classical	217

Name: count, dtype: int64

10-genre split counts:
(set, split)

test	2360
validation	2169
training	1800

Name: count, dtype: int64

In [9]:

```
# =====
# 9. CREATE CLEAN EXPORT DATAFRAME
# =====

pilot_10_export = pd.DataFrame({
    "track_id": pilot_10.index.astype(int),
    "genre_top": pilot_10[("track", "genre_top")].astype(str).values,
    "split": pilot_10[("set", "split")].astype(str).values,
    "audio_path": pilot_10["audio_path"].astype(str).values,
    "audio_exists": pilot_10["audio_exists"].astype(bool).values
})

print("10-genre export shape:", pilot_10_export.shape)
display(pilot_10_export.head())
```

10-genre export shape: (6329, 5)

	track_id	genre_top	split	audio_path	audio_exists
0	183	Rock	test	../data/raw/audio/fma_large\000\000183.mp3	True
1	184	Rock	test	../data/raw/audio/fma_large\000\000184.mp3	True
2	191	Folk	training	../data/raw/audio/fma_large\000\000191.mp3	True
3	205	Folk	training	../data/raw/audio/fma_large\000\000205.mp3	True
4	306	Rock	training	../data/raw/audio/fma_large\000\000306.mp3	True

In [10]:

```
# =====
# 10. SAVE 10-GENRE PILOT METADATA
# =====

os.makedirs("../data/processed", exist_ok=True)

pilot_10_export.to_csv(
    "../data/processed/audio_large_pilot_metadata_10genre.csv",
    index=False
)

print("Saved 10-genre pilot metadata to:")
print("../data/processed/audio_large_pilot_metadata_10genre.csv")
```

Saved 10-genre pilot metadata to:

../data/processed/audio_large_pilot_metadata_10genre.csv

In [11]:

```
# =====
# 11. INTERPRETATION NOTES
# =====

print("1. This notebook extends the validated 8-genre setup to a broader 10-genre pilot.")
print("2. International and Spoken were selected because they are the next most viable genres by available count.")
print("3. Caps were chosen conservatively so that all 10 genres remain balanced across train, validation, and test.")
print("4. The saved file can be used for 10-genre structured, audio, and hybrid experiments.")
```

1. This notebook extends the validated 8-genre setup to a broader 10-genre pilot.
2. International and Spoken were selected because they are the next most viable genres by available count.
3. Caps were chosen conservatively so that all 10 genres remain balanced across train, validation, and test.
4. The saved file can be used for 10-genre structured, audio, and hybrid experiments.